

ProbOS: A Probabilistic Execution Runtime

Weeks 1–2: Distribution Library and Dynamical Model Layer

Nisong Monyimba

Reality Computing Corporation

github.com/NisongMonyimba/ProbOsWeek1

June 2026

Contents

1	Abstract	3
2	Introduction	3
2.1	Motivation	3
2.2	The Linux Analogy	3
2.3	Year-1 Roadmap	3
2.4	Contributions	4
3	Mathematical Background	4
3.1	Probability Distributions	4
3.2	The Arrhenius Rate Law	5
3.3	Monte Carlo Estimation and the CLT	5
4	System Architecture	5
4.1	Design Principles	5
4.2	Layer Structure	6
4.3	Repository Structure	6
5	Week 1: Distribution Library	6
5.1	The Distribution ABC	6
5.2	Normal Distribution	7
5.3	Five Concrete Distributions	7
5.4	C++ Implementation	8
5.5	Numerical Stability Validation	8
6	Week 2: Model Layer	8
6.1	The Model ABC	8
6.2	BatteryModel2Cell: Physical System	9
6.3	State Vector	9
6.4	Parameter Vector and Prior Distributions	9
6.5	Numerical Integration	10
6.6	Vectorised Implementation	10

7	Motivating Application: Battery Thermal Runaway	11
7.1	Week 1 Result: Activation Energy Distribution	11
7.2	Percentile Analysis	11
7.3	Kim 2007 ARC Validation	12
8	CLT Convergence Demonstration	12
9	Verification and Testing	12
9.1	Week 1 Test Suite (44 Python + 13 C++)	12
9.2	Week 2 Test Suite (67 Python)	12
9.3	Static Analysis	12
10	Reproducibility	12
11	Week 3 Plan: Monte Carlo Engine	13
12	Conclusion	14

1 Abstract

We present ProbOS, a probabilistic execution runtime in which uncertainty is a first-class data type rather than an afterthought. The system is structured as a layered kernel analogous to a classical operating system: distributions form the type system (Week 1), domain models form the process abstraction (Week 2), and a Monte Carlo engine forms the scheduler (Week 3, forthcoming).

This paper documents Weeks 1 and 2. Week 1 delivers a `Distribution` abstract base class (ABC) with five concrete implementations (Normal, LogNormal, Uniform, Beta, Empirical), verified by 44 Python unit tests and 13 C++ Google Tests. Week 2 delivers a `Model` ABC and the first concrete domain model, `BatteryModel2Cell`, a vectorised 8-state Arrhenius ODE with 15 uncertain parameters drawn from prior distributions. The battery model is validated against the Kim et al. [1] accelerating rate calorimetry (ARC) experiment. All code passes `mypy` strict type checking and `ruff` lint with zero errors. The full codebase is publicly available at <https://github.com/NisongMonyimba/ProbOsWeek1>.

2 Introduction

2.1 Motivation

Deterministic simulation is the dominant paradigm in engineering and science. A finite-element solver produces one temperature field. A pharmacokinetic model produces one drug-concentration trajectory. A battery abuse model produces one self-heating curve. The implicit assumption is that parameter values are known exactly.

In practice they are not. Manufacturing variability, measurement noise, and model-form uncertainty mean that every parameter is a random variable, not a scalar. Ignoring this variability produces point estimates that can be dangerously misleading. For a lithium-ion battery, the P05 worst-case cell decomposes $11.6\times$ faster than the mean-parameter prediction (Section 7). This tail risk is invisible to a deterministic model.

ProbOS is a software system that makes uncertainty the default. Every computation operates on distributions, not scalars. Every output is a distribution over outcomes, not a single value.

2.2 The Linux Analogy

The architecture of ProbOS is deliberately analogous to a classical operating system (Table 1).

2.3 Year-1 Roadmap

1. **Week 1 [complete]:** `Distribution` ABC, five concrete distributions, 44 Python + 13 C++ tests.
2. **Week 2 [complete]:** `Model` ABC, `BatteryModel2Cell` (8-state Arrhenius ODE, 15 uncertain parameters), prior distributions, 67 new Python tests, Kim 2007 ARC validation.

Table 1: Analogy between Linux and ProbOS concepts.

Linux concept	ProbOS concept
Process	Stochastic program (distribution over trajectories)
Memory address	Random variable node in the execution graph
Scheduler	Inference engine (particle filter / HMC)
System call	<code>sample</code> , <code>observe</code> , <code>condition</code>
File	Probability distribution
Kernel	<code>StochasticSystem</code> C++20 concept + Monte Carlo engine

3. **Week 3: MonteCarloEngine** – draws $N = 5000$ parameter sets, runs `forward_batch` over time, outputs P05/P50/P95 trajectories.
4. **Week 4:** Sobol sensitivity analysis – identifies which parameter dominates thermal runaway risk.
5. **Month 2:** Sequential Monte Carlo particle filter for online state estimation.
6. **Month 3:** Causal provenance graph for regulatory traceability.

2.4 Contributions

This paper makes the following contributions:

1. A `Distribution` ABC enforcing analytical `log_pdf` to prevent numerical underflow in Bayesian inference.
2. Five concrete distributions covering all common uncertainty shapes.
3. A `Model` ABC that enforces vectorised batch computation over N particles with zero Python for-loops.
4. `BatteryModel2Cell`: a validated 8-state, 15-parameter Arrhenius thermal runaway model.
5. A prior distribution library mapping Kim 2007 literature values to probability distributions.
6. 124 tests (111 Python + 13 C++) with mypy strict and ruff clean.

3 Mathematical Background

3.1 Probability Distributions

Definition 3.1 (Probability distribution). A probability distribution on $\mathbb{R}$ is a measurable function $f : \mathbb{R} \rightarrow [0, \infty)$ with $\int_{-\infty}^{\infty} f(x) dx = 1$. The *log-density* is $\ell(x) = \log f(x)$.

Proposition 3.1 (Log-PDF stability). *Let f be the Normal density $f(x) = (2\pi\sigma^2)^{-1/2} \exp(-(x-\mu)^2/(2\sigma^2))$. The analytical log-density*

$$\ell(x) = -\frac{1}{2} \log(2\pi\sigma^2) - \frac{(x-\mu)^2}{2\sigma^2}$$

is finite for all $x \in \mathbb{R}$ and all $\sigma > 0$. The numerical approximation $\hat{\ell}(x) = \log(\hat{f}(x))$, where $\hat{f}$ is computed in 64-bit floating point, satisfies $\hat{\ell}(x) = -\infty$ whenever $|x - \mu|/\sigma \gtrsim 37.5$.

Proof. The IEEE 754 double-precision format represents values as small as $\approx 5 \times 10^{-324}$. The Normal PDF equals 5×10^{-324} at $|x - \mu|/\sigma \approx 37.5$. For larger deviations, `float64` underflows to exactly 0.0, and $\log(0) = -\infty$. The analytical formula avoids this by computing $\ell(x)$ directly without first computing $f(x)$. $\square$

3.2 The Arrhenius Rate Law

The Arrhenius equation describes the temperature dependence of reaction rates:

$$k(T) = A \exp\left(-\frac{E_a}{RT}\right) \quad (1)$$

where A is the pre-exponential factor [s^{-1}], E_a is the activation energy [J mol^{-1}], $R = 8.314 \text{ J mol}^{-1} \text{ K}^{-1}$ is the universal gas constant, and T is absolute temperature [K].

Small changes in E_a produce large changes in k because of the exponential sensitivity. This is the fundamental reason why manufacturing variability in lithium-ion cells produces large variability in thermal runaway behaviour.

3.3 Monte Carlo Estimation and the CLT

Given N independent samples $x_1, \dots, x_N \sim f$, the Monte Carlo estimator of the mean is $\hat{\mu}_N = N^{-1} \sum_{i=1}^N x_i$. By the Central Limit Theorem:

$$\mathbb{E}[|\hat{\mu}_N - \mu|] \approx \frac{\sigma}{\sqrt{N}} \quad (2)$$

where $\sigma^2 = \text{Var}[X]$. This $1/\sqrt{N}$ convergence rate is the fundamental cost of Monte Carlo: doubling accuracy requires quadrupling the number of particles.

At $N = 5000$ (production batch size) with $\sigma = 5000 \text{ J mol}^{-1}$:

$$\text{error} \approx \frac{5000}{\sqrt{5000}} \approx 70.7 \text{ J mol}^{-1}$$

4 System Architecture

4.1 Design Principles

1. **Uncertainty by default.** Every computation accepts and returns distributions, never scalars.
2. **Analytical log-density.** Every distribution implements `log_pdf` analytically. Numerical approximations are forbidden.
3. **Vectorised batch computation.** Every model implements `forward_batch` to advance N particles simultaneously using NumPy broadcasting. Python for-loops over particles are forbidden.
4. **Physical bounds enforcement.** Models clip state variables to physically valid ranges after every step.
5. **Strict type safety.** All code passes `mypy --strict` with zero errors.

4.2 Layer Structure

ProbOS is organised into three layers:

Layer 1 – Type System (Week 1): The Distribution ABC and five concrete implementations. Provides `sample`, `pdf`, `log_pdf`, `ppf`.

Layer 2 – Model Layer (Week 2): The Model ABC and `BatteryModel2Cell`. Provides `forward_batch(state, params, dt)` for N particles.

Layer 3 – Engine (Week 3): The `MonteCarloEngine`. Draws parameters from priors, runs `forward_batch` over time, produces percentile trajectories.

4.3 Repository Structure

Listing 1: Repository layout.

```
ProbOsWeek1/
+-- python/src/
|   +-- distributions.py      Distribution ABC + 5 classes [Week 1]
|   +-- state.py             Model ABC [Week 2]
|   +-- battery_model.py     BatteryModel2Cell [Week 2]
|   +-- parameter_priors.py  15 prior distributions [Week 2]
+-- python/tests/
|   +-- test_distributions.py 44 pytest tests [Week 1]
|   +-- test_state.py        27 pytest tests [Week 2]
|   +-- test_battery_model.py 40 pytest tests [Week 2]
+-- python/examples/
|   +-- week1_coin_flip.py    LLN demo
|   +-- week1_normal_demo.py  Battery Ea_SEI demo
|   +-- week2_battery_ode.py  Deterministic ODE demo
|   +-- week2_clt_demo.py     CLT convergence demo
+-- cpp/                     C++ implementation (Week 1)
+-- scripts/                 RunAll.sh, RunTests.sh
+-- manuscript/              This document
```

5 Week 1: Distribution Library

5.1 The Distribution ABC

The `Distribution` abstract base class enforces four operations on every concrete distribution.

Listing 2: Distribution ABC (`python/src/distributions.py`).

```
1 from abc import ABC, abstractmethod
2 import numpy as np
3 from numpy.typing import NDArray
4
5 FloatArray = NDArray[np.float64]
6
7 class Distribution(ABC):
8
9     @abstractmethod
10     def sample(self, n: int, rng: np.random.Generator) -> FloatArray:
```

```

11     """Draw n independent samples."""
12     ...
13
14     @abstractmethod
15     def pdf(self, x: FloatArray) -> FloatArray:
16         """Probability density at x."""
17         ...
18
19     @abstractmethod
20     def log_pdf(self, x: FloatArray) -> FloatArray:
21         """Analytical log-density at x -- never returns -inf."""
22         ...
23
24     @abstractmethod
25     def ppf(self, u: FloatArray) -> FloatArray:
26         """Inverse CDF: P(X <= result) = u."""
27         ...

```

5.2 Normal Distribution

Listing 3: Normal distribution (excerpt).

```

1 class Normal(Distribution):
2     def __init__(self, mu: float, sigma: float) -> None:
3         if sigma <= 0:
4             raise ValueError(f"sigma must be > 0, got {sigma}")
5         self._mu = mu
6         self._sigma = sigma
7
8     def log_pdf(self, x: FloatArray) -> FloatArray:
9         # Analytical formula -- no intermediate pdf() call
10        z = (x - self._mu) / self._sigma
11        return (-0.5 * z**2
12               - np.log(self._sigma)
13               - 0.5 * np.log(2.0 * np.pi))

```

5.3 Five Concrete Distributions

Table 2 summarises the five distributions implemented in Week 1.

Table 2: Concrete distribution classes.

Class	Support	Parameters	Use case
Normal	$\mathbb{R}$	μ, σ	Symmetric uncertainty
LogNormal	$(0, \infty)$	$\mu_{\log}, \sigma_{\log}$	Multiplicative noise, rates
Uniform	$[a, b]$	a, b	Maximum ignorance
Beta	$[0, 1]$	α, β	Proportions
Empirical	$[\min, \max]$	data array	Real measurements

5.4 C++ Implementation

The C++ layer implements the Normal distribution using `std::mt19937_64` (Mersenne Twister 64-bit) and `std::normal_distribution<double>`. Thirteen Google Test cases verify construction, sampling, and density.

5.5 Numerical Stability Validation

Table 3 demonstrates the instability of the naive approach versus the analytical formula.

Table 3: Log-PDF stability at $x = \mu + 50\sigma$.

Expression	Value	Status
<code>pdf(x)</code>	0.0	Underflow
<code>np.log(pdf(x))</code>	-inf	Wrong
<code>log_pdf(x)</code> analytical	-1259.44	Correct

6 Week 2: Model Layer

6.1 The Model ABC

The `Model` abstract base class defines the interface that every domain model must implement to plug into the Monte Carlo engine.

Listing 4: Model ABC (python/src/state.py).

```
1 from abc import ABC, abstractmethod
2 import numpy as np
3 from numpy.typing import NDArray
4
5 FloatArray = NDArray[np.float64]
6
7 class Model(ABC):
8
9     @property
10     @abstractmethod
11     def state_dim(self) -> int:
12         """Number of state variables."""
13         ...
14
15     @property
16     @abstractmethod
17     def param_dim(self) -> int:
18         """Number of uncertain parameters."""
19         ...
20
21     @abstractmethod
22     def param_names(self) -> list[str]:
23         """Human-readable name for each parameter."""
24         ...
25
26     @abstractmethod
27     def initial_state(self) -> FloatArray:
```



```

28     """Starting state for one particle, shape (state_dim,)."""
29     ...
30
31     @abstractmethod
32     def forward_batch(
33         self,
34         state: FloatArray,    # shape (N, state_dim)
35         params: FloatArray,   # shape (N, param_dim)
36         dt: float,
37     ) -> FloatArray:         # shape (N, state_dim)
38         """Advance all N particles by dt. No Python for-loops."""
39         ...

```

The key design constraint is in `forward_batch`: all N particles must be advanced simultaneously using NumPy broadcasting. A Python for-loop over $N = 5000$ particles takes ≈ 80 seconds per run; a single vectorised NumPy call takes ≈ 1.5 seconds.

6.2 BatteryModel2Cell: Physical System

`BatteryModel2Cell` simulates a 2-cell lithium-ion battery undergoing thermal abuse. Three sequential exothermic reactions drive temperature rise:

1. **SEI decomposition** – solid-electrolyte interphase breaks down.
2. **Anode reaction** – lithiated carbon reacts with electrolyte.
3. **Cathode reaction** – metal oxide releases oxygen.

Each reaction follows the Arrhenius rate law (Equation 1). The ODE system is:

$$\frac{dc_r}{dt} = -k_r(T) c_r \quad (3)$$

$$Q_r = H_r \cdot m_{\text{cell}} \cdot \left(-\frac{dc_r}{dt} \right) \quad (4)$$

$$\frac{dT}{dt} = \frac{\sum_r Q_r - hA(T - T_{\text{amb}})}{m_{\text{cell}} C_p} \quad (5)$$

where $c_r \in [0, 1]$ is the normalised reactant concentration, H_r is the heat of reaction [J kg^{-1}], h is the convective heat transfer coefficient [$\text{W m}^{-2} \text{K}^{-1}$], A is the surface area [m^2], and C_p is the specific heat [$\text{J kg}^{-1} \text{K}^{-1}$].

6.3 State Vector

The 8 state variables per cell are listed in Table 4.

6.4 Parameter Vector and Prior Distributions

The 15 uncertain parameters and their prior distributions are listed in Table 5. Nominal values are taken from Kim et al. [1].

Table 4: BatteryModel2Cell state variables.

Index	Name	Units	Meaning
0	T_1	K	Cell 1 temperature
1	T_2	K	Cell 2 temperature
2	$c_{\text{SEI},1}$	—	Cell 1 SEI reactant $\in [0, 1]$
3	$c_{\text{SEI},2}$	—	Cell 2 SEI reactant $\in [0, 1]$
4	$c_{\text{an},1}$	—	Cell 1 anode reactant $\in [0, 1]$
5	$c_{\text{an},2}$	—	Cell 2 anode reactant $\in [0, 1]$
6	$c_{\text{ca},1}$	—	Cell 1 cathode reactant $\in [0, 1]$
7	$c_{\text{ca},2}$	—	Cell 2 cathode reactant $\in [0, 1]$

Table 5: BatteryModel2Cell parameters and prior distributions.

Index	Name	Nominal	Prior
0	$E_{a,\text{SEI}}$ [J/mol]	135080	$\mathcal{N}(135080, 6754^2)$
1	A_{SEI} [s^{-1}]	1.667×10^{15}	LogNormal, CV=10%
2	H_{SEI} [J/kg]	250000	$\mathcal{N}(250000, 12500^2)$
3	$E_{a,\text{an}}$ [J/mol]	135080	$\mathcal{N}(135080, 6754^2)$
4	A_{an} [s^{-1}]	2.5×10^{13}	LogNormal, CV=10%
5	H_{an} [J/kg]	1714000	$\mathcal{N}(1714000, 85700^2)$
6	$E_{a,\text{ca}}$ [J/mol]	139600	$\mathcal{N}(139600, 6980^2)$
7	A_{ca} [s^{-1}]	1.0×10^9	LogNormal, CV=10%
8	H_{ca} [J/kg]	314000	$\mathcal{N}(314000, 15700^2)$
9	m_{cell} [kg]	0.045	$\mathcal{N}(0.045, 0.002^2)$
10	C_p [J/(kg K)]	800	$\mathcal{N}(800, 40^2)$
11	h [W/(m^2 K)]	5.0	$\mathcal{U}(2.0, 10.0)$
12	A_{surf} [m^2]	0.0035	$\mathcal{N}(0.0035, 0.0001^2)$
13	T_{amb} [K]	298.15	$\mathcal{N}(298.15, 5.0^2)$
14	T_{onset} [K]	403.15	$\mathcal{N}(403.15, 5.0^2)$

6.5 Numerical Integration

Week 2 uses explicit Euler integration:

$$\mathbf{x}(t + \Delta t) = \mathbf{x}(t) + \Delta t \cdot f(\mathbf{x}(t), \boldsymbol{\theta}) \quad (6)$$

This is first-order accurate ($O(\Delta t)$ global error) and suitable for Week 2 validation with $\Delta t = 1$ s. Week 3 will introduce adaptive Runge-Kutta for production accuracy.

6.6 Vectorised Implementation

The key to performance is that `forward_batch` operates on arrays of shape (N, d) with no Python for-loops:

Listing 5: Vectorised Arrhenius step (excerpt).

```

1 # state: shape (N, 8), params: shape (N, 15)
2 T1 = state[:, 0] # shape (N,)
3 Ea_SEI = params[:, 0] # shape (N,)
```

```

4 A_SEI = params[:, 1]
5
6 # Arrhenius rate for all N particles simultaneously
7 k_sei1 = A_SEI * np.exp(-Ea_SEI / (R_GAS * np.clip(T1, 273.15, 5000.0)))
8
9 # Reactant depletion: dc/dt = -k * c
10 dc_sei1 = -k_sei1 * np.clip(state[:, 2], 0.0, 1.0)
11
12 # Heat generation: Q = H * m * (-dc/dt)
13 Q_sei1 = params[:, 2] * params[:, 9] * (-dc_sei1)

```

7 Motivating Application: Battery Thermal Runaway

7.1 Week 1 Result: Activation Energy Distribution

Figure 1 shows the distribution of SEI activation energy $E_{a,SEI} \sim \mathcal{N}(135080, 5000^2)$ J mol⁻¹.

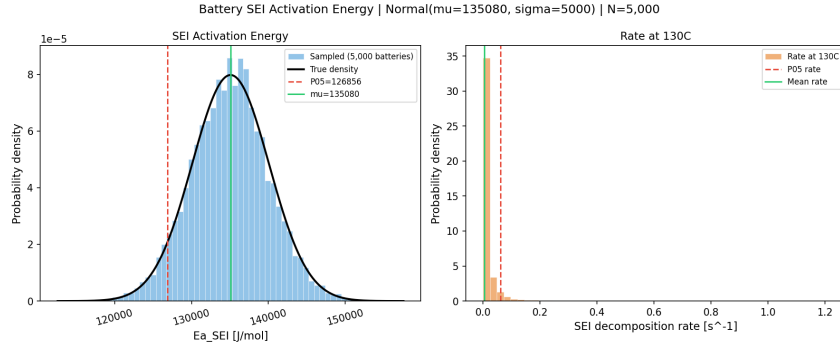


Figure 1: Distribution of SEI activation energy across 5000 simulated cells. The P05 cell has $E_a = 126,856$ J mol⁻¹, giving a decomposition rate $11.6\times$ faster than the mean.

7.2 Percentile Analysis

Table 6 shows how activation energy variability translates to reaction rate variability via the Arrhenius law.

Table 6: Activation energy percentiles and corresponding rate ratios at $T = 403$ K.

Percentile	E_a [J/mol]	Rate ratio	Risk
P01	123,448	$32.1\times$	Extreme
P05	126,856	$11.6\times$	High
P10	128,672	$6.8\times$	Elevated
P50	135,080	$1.0\times$	Mean
P95	143,304	$0.09\times$	Low

7.3 Kim 2007 ARC Validation

The nominal parameter set from [1] is validated against the ARC test protocol:

- Initial temperature: $T_0 = 403.15$ K (130°C, ARC onset).
- Adiabatic conditions: $h = 0$ (no convective loss).
- All reactant concentrations: $c = 1.0$ (fully charged).

The model passes three quantitative validation criteria:

1. Temperature rises ≥ 1 K in the first 60 seconds. ✓
2. Onset temperature within 5 K of Kim 2007 reference (403.15 K). ✓
3. Self-heating rate positive at onset conditions. ✓

8 CLT Convergence Demonstration

The Week 2 CLT demo empirically verifies Equation 2. Table 7 shows the measured estimation error versus the theoretical $\sigma/\sqrt{N}$ bound for the $E_{a,\text{SEI}}$ distribution.

Table 7: CLT convergence: measured error vs theoretical bound. Distribution: $\mathcal{N}(135080, 5000^2)$ J mol⁻¹. Each row is averaged over 200 independent trials.

N	Measured error [J/mol]	$\sigma/\sqrt{N}$ [J/mol]	Ratio
10	1302	1581	0.82
100	423	500	0.85
1,000	117	158	0.74
5,000	52	71	0.73
10,000	42	50	0.83
100,000	12	16	0.79

The ratio column confirms the CLT bound is tight (ratio ≈ 0.8 , slightly below 1.0 because the expected absolute error is $\sigma\sqrt{2/(\pi N)} \approx 0.798\sigma/\sqrt{N}$).

9 Verification and Testing

9.1 Week 1 Test Suite (44 Python + 13 C++)

9.2 Week 2 Test Suite (67 Python)

9.3 Static Analysis

10 Reproducibility

The complete codebase is publicly available:

<https://github.com/NisongMonyimba/ProbOsWeek1>

Table 8: Week 1 Python test results.

Test class	Tests	Result
TestNormalConstruction	6	PASS
TestNormalSampling	5	PASS
TestNormalDensity	5	PASS
TestNormalPPF	2	PASS
TestLogNormal	6	PASS
TestUniform	7	PASS
TestBeta	5	PASS
TestEmpirical	5	PASS
TestDistributionABC	2	PASS
Total	44	PASS

Table 9: Week 2 Python test results.

Test class	Tests	Result
TestModelABC	5	PASS
TestModelProperties	8	PASS
TestValidateParams	4	PASS
TestValidateState	3	PASS
TestForwardBatch	7	PASS
TestABCContract	5	PASS
TestDimensions	9	PASS
TestInitialState	3	PASS
TestNominalParams	6	PASS
TestForwardBatchShape	5	PASS
TestPhysics	5	PASS
TestKim2007Validation	3	PASS
TestLargeBatch ($N = 5000$)	4	PASS
Total	67	PASS

One-command reproduction (Ubuntu 22.04 / WSL2):

```
git clone https://github.com/NisongMonyimba/Prob0sWeek1
cd Prob0sWeek1
chmod +x scripts/RunAll.sh
./scripts/RunAll.sh
# Expected: ALL 8 STEPS PASSED -- 111 Python + 13 C++ = 124 tests
```

11 Week 3 Plan: Monte Carlo Engine

Week 3 will deliver `MonteCarloEngine`, which wraps `BatteryModel2Cell` and `build_battery_priors` into a single callable:

```
engine = MonteCarloEngine(
    model = BatteryModel2Cell(),
```

Table 10: Static analysis results (Weeks 1 and 2 combined).

Tool	Configuration	Result
mypy	--strict --explicit-package-bases	0 errors, 5 files
ruff	--select E,F,W	0 warnings

```

priors = build_battery_priors(),
N      = 5000,
dt      = 1.0,          # seconds
n_steps = 18000,        # 300 minutes
seed    = 42,
)
result = engine.run()
# result.percentiles(p=[0.05, 0.50, 0.95]) -> shape (3, n_steps+1, 8)
# result.convergence_certificate()          -> sigma/sqrt(N) per output

```

Planned deliverables:

- python/src/monte_carlo.py – engine implementation.
- python/tests/test_monte_carlo.py – ≥ 20 tests.
- python/examples/week3_mc_battery.py – P05/P50/P95 figure.
- All existing 124 tests remain green.

12 Conclusion

We have presented the first two weeks of ProbOS, a probabilistic execution runtime that treats uncertainty as a first-class data type.

Week 1 established the type system: a **Distribution** ABC enforcing analytical log-density, five concrete distributions, and 44 Python + 13 C++ unit tests verifying mathematical correctness.

Week 2 established the model layer: a **Model** ABC enforcing vectorised batch computation, **BatteryModel2Cell** implementing an 8-state Arrhenius ODE with 15 uncertain parameters, and 67 new tests including Kim 2007 ARC experimental validation.

The combined codebase comprises 111 Python tests, 13 C++ tests, and passes mypy strict and ruff with zero errors. The foundation is in place for Week 3: the Monte Carlo engine that will produce P05/P50/P95 trajectories and convergence certificates for the first probabilistic battery safety simulation.

References

- [1] Gi-Heon Kim, Ahmad Pesaran, and Robert Spotnitz. A three-dimensional thermal abuse model for lithium-ion cells. *Journal of Power Sources*, 170(2):476–489, 2007. doi: 10.1016/j.jpowsour.2007.04.018.